



Quiz Battle

PLATEFORME INTERACTIVE DE QUIZ FULLSTACK

Projet de Développement Web
Licence 2 Informatique — 2025/2026

Réalisé par :

BERRUEZO Arthur
VOLIANSKYI Nikita
MELKONIAN Mark

Sous la direction de :

M. Moulin

Table des matières

1	Introduction	2
2	Description du Sujet et Fonctionnalités	2
2.1	Le Hub : Point d'entrée de l'expérience	2
2.2	L'expérience de Jeu	3
2.3	Le Studio de Création	3
2.4	Mécaniques Avancées	3
3	Structure du Site et Architecture Logicielle	3
3.1	Architecture Frontend (React / TypeScript)	3
3.2	Architecture Backend (PHP Natif)	4
3.3	Communication Client-Serveur	4
4	Modèle de Données	5
4.1	Organisation des Tables	5
4.2	Usage du format JSON pour les Brouillons	5
4.3	Sécurité et Intégrité	5
5	Choix Techniques et Difficultés	6
5.1	L'adoption de TypeScript	6
5.2	Difficultés Rencontrées et Solutions	6
5.2.1	Communication Cross-Origin (CORS) et Proxy	6
5.2.2	Synchronisation et "Pseudo Temps Réel"	6
5.2.3	Le Studio de Création : Complexité de l'État	7
6	Répartition des Tâches	7
7	Conclusion	8
7.1	Bilan Technique	8
7.2	Perspectives	8

1 Introduction

Dans le cadre de la deuxième année de licence Informatique, le projet de développement web constitue une étape clé pour synthétiser l'apprentissage des technologies client-serveur. L'objectif imposé est la réalisation d'un site web dynamique complet, intégrant une gestion de base de données et une logique métier robuste.

Le projet que nous présentons, intitulé **Quiz Battle**, est une plateforme interactive dédiée à l'apprentissage et au divertissement par le biais de quiz thématiques. L'ambition de ce projet dépasse la simple création de pages statiques ; il s'agit d'une *Single Page Application* (SPA) moderne, offrant une fluidité de navigation similaire à une application native.

L'enjeu technique majeur a été de faire cohabiter deux environnements distincts :

- Un **Frontend** développé avec **React** et **TypeScript**, garantissant une interface utilisateur réactive et un code fortement typé.
- Un **Backend** en **PHP natif**, structuré pour répondre aux requêtes via une API RESTful, assurant la sécurité des données et la communication avec une base de données **MySQL**.

Ce rapport détaille les choix d'architecture, la structure de la base de données, ainsi que les défis techniques rencontrés lors de la mise en œuvre des fonctionnalités telles que le système de jeu, le studio de création et la gestion de l'authentification.

2 Description du Sujet et Fonctionnalités

L'application **Quiz Battle** a été conçue comme une plateforme communautaire et compétitive. Elle s'articule autour de trois piliers : l'exploration de contenu, l'expérience de jeu et la création.

2.1 Le Hub : Point d'entrée de l'expérience

Le "Hub" constitue la page d'accueil de l'utilisateur connecté. Contrairement à une liste statique, il est organisé pour favoriser la découverte :

- **Navigation par catégories** : Un système de carrousel interactif permet de filtrer les quiz par thématiques (Sciences, Cinéma, Histoire, etc.), facilitant l'accès aux sujets de prédilection de l'utilisateur.
- **Section Discover** : Cette zone met en avant des quiz variés et populaires pour inciter l'utilisateur à jouer immédiatement à de nouveaux contenus.
- **Système de recherche et filtrage** : Une barre de recherche dynamique permet de filtrer l'intégralité du catalogue par mots-clés en temps réel, garantissant une ergonomie optimale même avec un volume important de données.

2.2 L'expérience de Jeu

Le cœur de l'application est son moteur de jeu. Deux modes principaux ont été implémentés :

- **Le Mode Solo** : Un parcours d'entraînement où l'utilisateur répond à une série de questions. Une attention particulière a été portée au feedback visuel (barres de progression, animations de réussite) pour rendre l'expérience gratifiante.
- **Le Mode Lobby (Multi-joueurs)** : Un système de salons privés. Un utilisateur peut créer une salle et obtenir un code unique à 6 caractères. Ses amis peuvent alors rejoindre le salon via ce code pour une session de jeu synchronisée.

2.3 Le Studio de Création

Pour assurer la pérennité du contenu, un "Studio" complet a été développé. Il permet à tout utilisateur de devenir créateur :

- **Éditeur de Quiz** : Une interface dédiée à la gestion des questions, des réponses et des paramètres de jeu (temps par question, points).
- **Gestion des Brouillons (Drafts)** : Grâce à une structure de données flexible, les créateurs peuvent sauvegarder leur travail en cours sans obligation de publication immédiate. Ces brouillons sont stockés de manière complexe en base de données pour être repris à tout moment.

2.4 Mécaniques Avancées

Afin d'ajouter une dimension stratégique, un système d'inventaire d'objets a été intégré. Les joueurs peuvent obtenir et utiliser des items (bonus ou malus) qui influencent le déroulement de la partie, renforçant ainsi l'aspect "Battle" de la plateforme.

3 Structure du Site et Architecture Logicielle

L'un des défis majeurs de ce projet a été de concevoir une architecture capable de séparer la logique métier de l'interface utilisateur, tout en garantissant une maintenance aisée.

3.1 Architecture Frontend (React / TypeScript)

Le choix de **React** combiné à **TypeScript** a permis de transformer le site en une véritable application fluide. Nous avons appliqué plusieurs principes de design (*Design Patterns*) pour structurer le code :

- **Le Pattern Repository** : Pour ne pas rendre l'interface dépendante de l'API, nous avons créé une couche d'abstraction. Les composants utilisent des interfaces (*IQuizRepository*, *IGameRepository*). Cela nous a permis, durant le développement, de travailler avec des données simulées (*Mocks JSON*) avant de connecter le backend définitif.
- **Mappers de données** : Une couche de transformation a été mise en place pour convertir les données brutes provenant de MySQL (souvent en *snake_case*) vers des objets TypeScript normalisés en *camelCase*. Cela assure une cohérence parfaite sur l'ensemble du client.
- **Gestion d'état avec Hooks** : La logique de jeu, complexe par nature, a été isolée dans des *Custom Hooks* (ex : *useSoloGame*). Cette approche permet de séparer le calcul du score et le timing de l'affichage visuel des composants.

3.2 Architecture Backend (PHP Natif)

Le serveur a été conçu comme une **API RESTful** robuste. Contrairement à une approche PHP classique mélangeant HTML et logique, notre backend ne sert que du contenu de type `application/json`.

- **Système de Contrôleurs** : Le code est segmenté par responsabilités (*AuthController*, *QuizController*, *RoomController*). Chaque contrôleur interagit avec la base de données via **PDO**, garantissant ainsi la protection contre les injections SQL grâce aux requêtes préparées.
- **Authentification Stateless (JWT)** : Pour sécuriser les échanges sans utiliser les sessions PHP traditionnelles (peu adaptées aux SPA), nous avons implémenté un système de **JSON Web Tokens**. Le serveur signe un jeton lors de la connexion, que le client renvoie dans les headers de chaque requête.
- **Format de réponse unifié** : Toutes les sorties du serveur passent par une fonction utilitaire garantissant une structure de réponse constante (statut, message, données), facilitant ainsi le traitement des erreurs côté Frontend.

3.3 Communication Client-Serveur

La communication repose sur la bibliothèque **Axios**. Un intercepteur a été configuré pour injecter automatiquement le token JWT dans les headers de chaque requête sortante et pour rediriger l'utilisateur vers la page de connexion en cas d'expiration de sa session (erreur 401).

4 Modèle de Données

La persistance des données est assurée par un système de gestion de base de données relationnelle **MySQL**. La structure a été pensée pour être évolutive, avec un total de 12 tables organisées par domaines de responsabilité.

4.1 Organisation des Tables

Le schéma relationnel peut être divisé en quatre grands pôles :

- **Utilisateurs et Progression** : Les tables `users` et `user_stats` gèrent non seulement l'authentification (mots de passe hachés), mais aussi le suivi de l'expérience (XP), du niveau et des victoires.
- **Contenu des Quiz** : Une hiérarchie stricte est respectée : `categories` → `quizzes` → `questions` → `answers`. Cette structure permet une grande flexibilité dans la gestion des thématiques.
- **Moteur de Jeu et Sessions** : Les tables `game_sessions` et `game_players` stockent l'état vivant d'une partie (en attente, en cours, terminée). C'est ici que sont gérés les scores en temps réel et les codes de salon uniques.
- **Système d'Inventaire et Pénalités** : Les tables `items`, `player_inventory` et `active_penalties` préparent le terrain pour les mécaniques de "Battle", permettant de traquer quel joueur possède quel objet et quel malus est appliqué à qui.

4.2 Usage du format JSON pour les Brouillons

Une particularité technique de notre modèle réside dans la table `quiz_drafts`. Pour le "Studio", nous avons fait le choix d'utiliser le type **JSON** pour la colonne `content`.

Ce choix s'explique par la nature flexible d'un brouillon : un utilisateur peut modifier radicalement la structure de ses questions sans que nous ayons besoin de valider l'intégrité relationnelle complète avant la publication finale. Cela simplifie considérablement les échanges entre le Frontend (qui manipule des objets JSON) et le Backend.

4.3 Sécurité et Intégrité

L'intégrité des données est garantie par l'utilisation rigoureuse de clés étrangères avec des contraintes `ON DELETE CASCADE`. Par exemple, la suppression d'un utilisateur entraîne automatiquement le nettoyage de ses statistiques et de son inventaire, évitant ainsi la présence de données orphelines. Enfin, l'accès aux données depuis PHP se fait exclusivement via **PDO** avec des paramètres liés, rendant les attaques par injection SQL impossibles.

5 Choix Techniques et Difficultés

Le développement d'une application de cette envergure impose des choix technologiques structurants pour répondre aux contraintes de temps et de qualité.

5.1 L'adoption de TypeScript

Le passage de JavaScript à **TypeScript** a été l'un des choix les plus bénéfiques du projet.

- **Sécurité du code** : Le typage strict des entités (ex : l'interface `GameSession` ou `User`) a permis d'éliminer une grande partie des erreurs de logique avant même l'exécution du code.
- **Auto-complétion et Maintenance** : Dans un projet comportant plus de 100 fichiers, la capacité de l'IDE à comprendre la structure des objets facilite grandement le *refactoring* et la collaboration.

5.2 Difficultés Rencontrées et Solutions

5.2.1 Communication Cross-Origin (CORS) et Proxy

Durant la phase de développement, le frontend (servi par Vite sur le port 5173) et le backend (servi par PHP sur le port 8000) sont considérés comme deux origines différentes par le navigateur. Cela a initialement bloqué toutes les requêtes API.

Solution : Plutôt que d'ouvrir totalement les politiques CORS (ce qui est peu sécurisé), nous avons configuré un **Proxy inverse** dans le fichier `vite.config.ts`. Ce dernier redirige de manière transparente les appels `/api` vers le serveur PHP, simulant ainsi une origine unique.

5.2.2 Synchronisation et "Pseudo Temps Réel"

L'absence de WebSockets (non couverts dans le cadre du cours de PHP natif) a rendu complexe la gestion des salons multi-joueurs (*Lobbies*). Comment savoir si un joueur a rejoint la salle sans que le serveur ne puisse "pousser" l'information au client ?

Solution : Nous avons implémenté une logique de **Polling** (appels récurrents à intervalles réguliers) côté client, couplée à une gestion d'états très stricte en base de données. Bien que plus gourmande en ressources qu'une connexion par socket, cette méthode assure une compatibilité parfaite avec un hébergement PHP standard tout en offrant une expérience utilisateur fluide.

5.2.3 Le Studio de Création : Complexité de l'État

Le Studio de création est la partie la plus complexe du Frontend. Gérer l'ajout, la suppression, la réorganisation des questions et la validation des réponses en temps réel demande une gestion d'état avancée.

Solution : Nous avons utilisé le hook `useReducer` pour centraliser la logique de modification du quiz. Cela permet de garantir que chaque action de l'utilisateur (ex : "Définir la bonne réponse") produise un état prévisible et facile à sauvegarder sous forme de brouillon.

6 Répartition des Tâches

Conformément aux modalités du projet, la charge de travail a été répartie entre les membres du groupe afin de couvrir l'ensemble du spectre "Fullstack". Bien que nous ayons collaboré sur les choix d'architecture globaux, les responsabilités principales ont été définies comme suit :

- **BERRUEZO Arthur :**
 - **Architecture Frontend :** Configuration de l'environnement de développement sous Vite et structuration globale du projet React en Single Page Application.
 - **Logique Métier Client :** Implémentation des mécanismes de gestion d'état complexe via les Hooks (*useState*, *useEffect*) et sécurisation des routes côté client.
 - **Moteurs Applicatifs :** Développement intégral du moteur de jeu Solo (gestion du cycle de vie des questions et calcul du score) et du moteur de gestion du Studio de création (manipulation dynamique des données du quiz).
- **VOLIANSKYI Nikita :**
 - **Infrastructure Backend :** Conception de l'architecture serveur en PHP natif et mise en place d'un système de routage modulaire pour l'API REST.
 - **Gestion des Ressources :** Développement des contrôleurs métier assurant la distribution des données de quiz, la gestion des catégories et le traitement des requêtes HTTP.
 - **Sécurisation Stateless :** Implémentation du système d'authentification par JSON Web Tokens (JWT), incluant la signature HMAC-SHA256 et la vérification des headers d'autorisation.
- **MELKONIAN Mark :**
 - **Ingénierie des Données :** Modélisation du schéma MySQL (12 tables) et optimisation des requêtes via PDO.
 - **Logique Backend Studio :** Développement des contrôleurs dédiés à la créa-

tion et à l'édition des quiz.

- **Gestion des Brouillons** : Conception d'un stockage hybride en base de données via le type JSON.

7 Conclusion

La réalisation de **Quiz Battle** a été une expérience particulièrement formatrice. Ce projet nous a permis de mettre en pratique les fondamentaux du développement web (HTML, CSS, PHP, SQL) tout en nous confrontant aux réalités du développement moderne avec React et TypeScript.

7.1 Bilan Technique

L'objectif initial de créer une plateforme robuste et sécurisée a été atteint. L'utilisation d'une architecture découplée (API REST d'un côté, SPA de l'autre) s'est avérée être un choix judicieux, facilitant le débogage et permettant une évolution indépendante des deux parties du projet. La rigueur imposée par TypeScript et l'organisation en "Pattern Repository" nous ont permis de livrer un code structuré, proche des standards de l'industrie informatique.

7.2 Perspectives

Le projet a été conçu avec une vision à long terme. Plusieurs pistes d'amélioration sont envisageables :

- **WebSockets** : Remplacer le système de *polling* par une connexion persistante pour un mode multijoueur en temps réel instantané.
- **Gamification** : Étendre le système d'inventaire et introduire des classements mondiaux (Leaderboards).
- **Accessibilité** : Améliorer la conformité aux normes WCAG pour rendre la plateforme utilisable par le plus grand nombre.

En conclusion, ce projet a non seulement validé nos compétences techniques en L2 Informatique, mais il nous a également appris à gérer la complexité d'une application complète, de la modélisation des données jusqu'à l'expérience utilisateur finale.